

Little Tree Ventures Hub — schema proposal + next steps from today's call

5 messages

Tiffanie Rothwell <tiffanie@mjmventures.ai>

To: Andre Boudreault <andre.guy.b@gmail.com>, Mike David <mikedavidamg@gmail.com>

Tue, May 5, 2026 at 2:06 PM

Hey Andre,

First, thank you. I know you stole a day and a half from yourself in the middle of a year-end audit to get the Supabase stack live, and I don't take that lightly. Doing all of the heavy lifting... That's the kind of head start most people don't get, and we're starting from a real foundation because of you. Genuinely, thank you. And thank you for the consistent support along the way. This would not be possible without you!!

Today's call with Valera was a real pivot point for me. I had a 5-week plan I was holding tight and a bunch of assumptions Claude spit out that I hadn't pressure-tested with someone who actually builds these things. Walking out of that meeting I had way more clarity, and a much more honest plan. So here's where we landed and what I need from you to keep moving.

THE PROJECT IN ONE PARAGRAPH

We're building one brain for Mike across all his ventures — Little Tree Ventures (Little Tree Gas, LT Capital, LT Pay, LT Fuel, Ai Agency, Jayi, and more), with MedBox, KCF, Lahore, and MJM coming after. The hub pulls in his calendar, his Fireflies meetings, his Telegram messages, his email threads, his voice memos, and the KPIs from each business. At 6am he gets a Telegram brief: meetings today, prep notes, KPI pulse versus yesterday and versus a year ago, blockers, decisions waiting on him. He goes through the day, every meeting auto-captures, tasks get pushed to a Notion board, and by close of day the dashboard reflects everything that moved. Mike's hub is the proof of concept — once it works for him, we replicate the same pattern for the other CEOs. The intelligence layer (Claude API, pgvector for the mentor PDFs, Whisper for voice) is what makes it feel like a brain instead of a dashboard.

WHAT WE AGREED ON TODAY

A few decisions came out of the call that shifted my plan:

- Two databases, not one. Littletree (your existing MySQL) stays read-only, nightly midnight refresh, untouched. We build a second Supabase Postgres database called Little Tree Ventures for everything outside the transactional layer — meetings, calendar events, tasks, KPIs, briefs, all of it. Your call, and it's the right one.
- Build the schema service-by-service. Valera was clear: don't try to design the whole schema upfront. Add tables as each automation gets wired. So that's how I structured the proposal.
- Drop Google SSO from Week 1. It's a paid feature Supabase only ships on the cloud version, and it's not worth fighting on day one. We'll use Supabase email/password with manual user provisioning, and revisit SSO later if Mike wants Google Workspace tied in. Valera said "always a way to do it later" and I trust that.
- No dev/test database yet. You and Valera both landed on "build fast, regroup if and when we hit a scenario that needs it." Nothing's in production, so nothing to protect yet.
- Timeline compressed from 5 weeks to 3. Valera glanced at Week 1 and said it was a 1–2 day job, not 5. So I rebuilt the calendar — same scope, tighter cadence. It's all in the project-management dashboard if you want to see it.

THE SCHEMA PROPOSAL — YOUR NEXT STEP

Per your suggestion at the end of the meeting ("let Claude say what kind of Supabase database we need outside of the transactional stuff"), I had Claude run through the full project plan and propose a first-pass schema. It's saved as mikes-world-schema.sql. Here's the shape of it:

- 25 tables, grouped into 7 service blocks (foundation, ingestion, workflow, intelligence, cross-venture, mentors/RAG, snapshots)
- 10 migration blocks at the bottom of the file (M001 through M010) so you can apply it in clean steps via the Supabase CLI
- RLS policies wired in — there's a has_company_access(company_id) helper that every per-company table uses, so access rules live in one place
- The companies table is seeded with all 10 entities (Little Tree Gas, LT Capital, LT Pay, LT Fuel, Ai Agency, Jayi, MJM, MedBox, KCF, Lahore) so it runs as-is
- pgvector(1536) for the mentor PDF semantic search (matches OpenAI's text-embedding-3-small)
- A "Phase 3 deferred" comment block at the bottom listing tables we know we'll need eventually (email threads, SOPs, Stripe sync, QuickBooks log) but won't create until the matching automation is being wired

I'd love it if you could review the foundation block first — M001 and M002 — that's profiles, companies, company_access, auth_audit, and activity_log. If that lands cleanly, the rest follows the same pattern and we can apply each migration as I work through the corresponding service.

WHAT I NEED FROM YOU

- Review mikes-world-schema.sql, especially M001 and M002. Tell me where I'm wrong or where the design will bite us later.
- Confirm I have the right connection info for Studio and the read-only Littletree user. You sent it but I want to make sure I'm hitting the right endpoints before I start writing migrations against it.
- Confirm Mike's World goes on the same server, same Supabase instance as a separate database — or whether you'd rather it be its own instance. Either works for me, you know the operational side better than I do.
- Whenever you have 20 minutes, a quick walk-through of the storage bucket setup so I don't fumble the PDF and voice memo paths.

No rush on any of this. I appreciate any time you put to this. Whenever you can get to it.

Thank you again for everything, Andre!!!!

Talk soon,
Tiffanie—

Tiffanie Rothwell
Project Manager
Little Tree Ventures
E: tiffanie@mjmventures.ai
[Schedule a meeting with me here](#)

andre.guy.b@gmail.com <andre.guy.b@gmail.com>

To: Tiffanie Rothwell <tiffanie@mjmventures.ai>, Mike David <mikedavidamg@gmail.com>

Tue, May 5, 2026 at 2:10 PM

Hi,

Missing this Review mikes-world-schema.sql

André Boudreault

514 647-7577

[Quoted text hidden]

Tiffanie Rothwell <tiffanie@mjmventures.ai>

To: andre.guy.b@gmail.com

Cc: Mike David <mikedavidamg@gmail.com>

Tue, May 5, 2026 at 2:20 PM

In case it's easier to download from the google drive: https://drive.google.com/file/d/1WpVNLr25b_RApnffXMyymYB1w7xbW7q4/view?usp=sharing

Here is a copy/paste:

—

— MIKE'S WORLD · Supabase Schema Proposal

— Self-hosted Supabase (Postgres 15) on Andre's server

— Sits ALONGSIDE the read-only Littletree MySQL transactional DB.

— Built service-by-service per Valera's recommendation — Phase 1 first,

— Phase 2 added as automations get wired, Phase 3 deferred until needed.

—

— Conventions:

— · Every table has id (uuid), created_at, updated_at

— · Every table that is per-company has company_id with RLS

— · created_by references auth.users(id) where applicable

— · Soft delete via deleted_at where edits matter (meetings, tasks, decisions)

— · JSONB for flexible payloads we don't want to over-design upfront

— · Indexes added on every FK + every column we filter by often

—

— Required extensions

CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE EXTENSION IF NOT EXISTS vector; -- pgvector, for mentor PDF search

-- PHASE 1 - FOUNDATION (Week 1, Mon-Tue)
 -- Auth, companies, RLS scaffold, universal activity log.
 -- Nothing else can be wired until these exist.

-- 01 -- profiles --
 -- Every auth.users row gets a profile. Role drives post-login redirect.
 CREATE TABLE profiles (
 id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
 email text UNIQUE NOT NULL,
 full_name text,
 role text NOT NULL CHECK (role IN ('ceo','manager','employee','admin')),
 theme text DEFAULT 'dark' CHECK (theme IN ('dark','light')),
 telegram_id text, -- chat_id for CommandOS Dms
 avatar_url text,
 created_at timestampz NOT NULL DEFAULT now(),
 updated_at timestampz NOT NULL DEFAULT now()
);
 ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
 CREATE POLICY profiles_self_read ON profiles FOR SELECT USING (auth.uid() = id);
 CREATE POLICY profiles_self_update ON profiles FOR UPDATE USING (auth.uid() = id);
 CREATE POLICY profiles_admin_all ON profiles FOR ALL USING (
 EXISTS (SELECT 1 FROM profiles p WHERE p.id = auth.uid() AND p.role IN ('ceo','admin'))
);

-- 02 -- companies --
 -- The 5 LTV ventures + MJM, MedBox, KCF, Lahore. Anchor for RLS.
 CREATE TABLE companies (
 id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
 slug text UNIQUE NOT NULL, -- 'little-tree-gas', 'lt-capital', etc.
 name text NOT NULL,
 parent_group text NOT NULL CHECK (parent_group IN ('ltv','mjm','medbox','kcf','lahore')),
 external_url text, -- existing dashboard URL if any
 is_active boolean NOT NULL DEFAULT true,
 created_at timestampz NOT NULL DEFAULT now(),
 updated_at timestampz NOT NULL DEFAULT now()
);
 INSERT INTO companies (slug, name, parent_group) VALUES
 ('little-tree-gas','Little Tree Gas','ltv'),
 ('lt-capital','LT Capital','ltv'),
 ('lt-pay','LT Pay','ltv'),
 ('lt-fuel','LT Fuel','ltv'),
 ('ai-agency','Ai Agency','ltv'),
 ('jayi','Jayi','ltv'),
 ('mjm','MJM Ventures','mjm'),
 ('medbox','MedBox','medbox'),
 ('kcf','KCF','kcf'),
 ('lahore','Lahore','lahore');

-- 03 -- company_access --
 -- Many-to-many: which users can see which companies + at what permission level.
 CREATE TABLE company_access (
 id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
 user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
 company_id uuid NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
 permission text NOT NULL CHECK (permission IN ('read','write','admin')),
 created_at timestampz NOT NULL DEFAULT now(),
 UNIQUE (user_id, company_id)
);
 CREATE INDEX idx_company_access_user ON company_access(user_id);
 CREATE INDEX idx_company_access_company ON company_access(company_id);
 ALTER TABLE company_access ENABLE ROW LEVEL SECURITY;
 CREATE POLICY company_access_self_read ON company_access FOR SELECT USING (auth.uid() = user_id);
 -- Helper: returns true if current user has access to company_id.
 CREATE OR REPLACE FUNCTION has_company_access(target_company uuid)
 RETURNS boolean LANGUAGE sql STABLE SECURITY DEFINER AS \$\$
 SELECT EXISTS (
 SELECT 1 FROM company_access
 WHERE user_id = auth.uid() AND company_id = target_company
) OR EXISTS (
 SELECT 1 FROM profiles WHERE id = auth.uid() AND role IN ('ceo','admin')
);
 \$\$;

-- 04 -- auth_audit --
 -- Every login, logout, session refresh, failed attempt. Telegram on anomaly.
 CREATE TABLE auth_audit (
 id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
 user_id uuid REFERENCES auth.users(id) ON DELETE SET NULL,
 event_type text NOT NULL CHECK (event_type IN ('login','logout','refresh','failed','password_reset')),
 ip_address inet,
 user_agent text,
 metadata jsonb DEFAULT '{}':jsonb,
 created_at timestampz NOT NULL DEFAULT now()
);
 CREATE INDEX idx_auth_audit_user_time ON auth_audit(user_id, created_at DESC);

-- 05 -- activity_log --
 -- Universal append-only log. Every automation, page load, KPI write lands here.
 -- Powers the cross-venture activity ticker (#138) and weekly synthesis.
 CREATE TABLE activity_log (
 id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
 user_id uuid REFERENCES auth.users(id) ON DELETE SET NULL,
 company_id uuid REFERENCES companies(id) ON DELETE SET NULL,
 source text NOT NULL, -- 'fireflies','gcal','telegram','manual','cron','claudie'
 event_type text NOT NULL, -- 'meeting_synced','task_created','kpi_updated', etc.
 entity_type text, -- 'meeting','task','decision', etc.
 entity_id uuid,
 summary text, -- one-line human-readable
 payload jsonb DEFAULT '{}':jsonb,
 created_at timestampz NOT NULL DEFAULT now()
);
 CREATE INDEX idx_activity_company_time ON activity_log(company_id, created_at DESC);
 CREATE INDEX idx_activity_user_time ON activity_log(user_id, created_at DESC);
 CREATE INDEX idx_activity_source ON activity_log(source, created_at DESC);

-- PHASE 1 - INGESTION (Week 1, Wed-Thu)
 -- Land the data Mike's hub depends on. All cron-pulled or webhook-pushed.

-- 06 -- calendar_events --
 -- Pulled from Mike's Google Calendar (cron, ~15 min cadence). Calendly + Zoom
 -- write here too via webhook -- upsert by external_id.
 CREATE TABLE calendar_events (
 id uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
 user_id uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
 company_id uuid REFERENCES companies(id) ON DELETE SET NULL,
 source text NOT NULL CHECK (source IN ('gcal','calendly','zoom','manual')),
 external_id text NOT NULL, -- gcal event id, calendly invite id
 title text NOT NULL,
 description text,
 location text,
 starts_at timestampz NOT NULL,
 ends_at timestampz NOT NULL,

```
attendees    jsonb DEFAULT '[]'::jsonb, -- [[email, name, response]]
meeting_url  text,
brief_sent   boolean NOT NULL DEFAULT false, -- T-24h Telegram brief flag
is_cancelled boolean NOT NULL DEFAULT false,
raw          jsonb, -- full provider payload for replay
created_at   timestamptz NOT NULL DEFAULT now(),
updated_at   timestamptz NOT NULL DEFAULT now(),
UNIQUE (source, external_id)
);
CREATE INDEX idx_cal_user_time ON calendar_events(user_id, starts_at);
CREATE INDEX idx_cal_company_time ON calendar_events(company_id, starts_at);
CREATE INDEX idx_cal_brief_pending ON calendar_events(brief_sent, starts_at) WHERE brief_sent = false;

-- 07 ----- meetings -----
-- Fireflies webhook lands here. Joined to calendar_events when possible.
CREATE TABLE meetings (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  calendar_event_id uuid REFERENCES calendar_events(id) ON DELETE SET NULL,
  company_id   uuid REFERENCES companies(id) ON DELETE SET NULL,
  source       text NOT NULL DEFAULT 'fireflies' CHECK (source IN ('fireflies','manual','zoom')),
  external_id  text NOT NULL,
  title        text NOT NULL,
  occurred_at  timestamptz NOT NULL,
  duration_seconds int,
  transcript_url text,
  recording_url text,
  summary      text, -- Fireflies AI summary
  action_items jsonb DEFAULT '[]'::jsonb,
  raw_transcript text,
  raw          jsonb,
  created_at   timestamptz NOT NULL DEFAULT now(),
  UNIQUE (source, external_id)
);
CREATE INDEX idx_meetings_company_time ON meetings(company_id, occurred_at DESC);
CREATE INDEX idx_meetings_event ON meetings(calendar_event_id);

-- 08 ----- meeting_attendees -----
CREATE TABLE meeting_attendees (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  meeting_id  uuid NOT NULL REFERENCES meetings(id) ON DELETE CASCADE,
  email       text NOT NULL,
  full_name   text,
  user_id     uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  is_internal boolean NOT NULL DEFAULT false,
  speaking_time_seconds int,
  created_at  timestamptz NOT NULL DEFAULT now(),
  UNIQUE (meeting_id, email)
);
CREATE INDEX idx_attendees_email ON meeting_attendees(email);
CREATE INDEX idx_attendees_user ON meeting_attendees(user_id);

-- 09 ----- telegram_messages -----
-- Every inbound (CommandOS user message) and outbound (brief, alert) message.
CREATE TABLE telegram_messages (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  direction   text NOT NULL CHECK (direction IN ('inbound','outbound')),
  chat_id     text NOT NULL,
  message_id  text, -- Telegram's id, for edits
  message_type text NOT NULL CHECK (message_type IN ('text','voice','photo','document','command')),
  body        text,
  voice_url   text, -- if voice memo, link to storage
  intent      text, -- 'capture','ask_mentor','status', resolved by router
  meeting_id  uuid REFERENCES meetings(id) ON DELETE SET NULL,
  task_id     uuid, -- FK added below after tasks table
  raw        jsonb,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_tg_user_time ON telegram_messages(user_id, created_at DESC);
CREATE INDEX idx_tg_chat ON telegram_messages(chat_id, created_at DESC);

-- PHASE 1 · WORKFLOW (Week 1 Fri → Week 2 Mon)
-- Tasks, decisions, capture, voice memos. The day-to-day surface.

-- 10 ----- tasks -----
-- Synced bidirectionally with the Notion task board (post-meeting decision).
CREATE TABLE tasks (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  company_id  uuid REFERENCES companies(id) ON DELETE SET NULL,
  meeting_id  uuid REFERENCES meetings(id) ON DELETE SET NULL,
  notion_page_id text UNIQUE, -- mirror of Notion page
  title       text NOT NULL,
  description text,
  assigned_to uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  assigned_to_label text, -- 'Tiffanie','Leah','Mike' for non-users
  status      text NOT NULL DEFAULT 'open' CHECK (status IN ('open','in_progress','blocked','done','cancelled')),
  priority    text CHECK (priority IN ('low','medium','high','urgent')),
  due_at      timestamptz,
  completed_at timestamptz,
  created_by  uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  source      text NOT NULL DEFAULT 'manual' CHECK (source IN ('manual','meeting','telegram','voice','notion')),
  deleted_at  timestamptz,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_tasks_assignee_status ON tasks(assigned_to, status);
CREATE INDEX idx_tasks_company_status ON tasks(company_id, status);
CREATE INDEX idx_tasks_due ON tasks(due_at) WHERE status NOT IN ('done','cancelled');

-- now we can wire tg → tasks
ALTER TABLE telegram_messages
ADD CONSTRAINT fk_tg_task FOREIGN KEY (task_id) REFERENCES tasks(id) ON DELETE SET NULL;

-- 11 ----- decision_queue -----
-- Surfaced on Mike's hub. Anything blocking, awaiting his call.
CREATE TABLE decision_queue (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  company_id  uuid REFERENCES companies(id) ON DELETE SET NULL,
  title       text NOT NULL,
  context     text, -- the "why now" -- Claude-written or human
  options     jsonb DEFAULT '[]'::jsonb, -- [[label, recommended_by, tradeoffs]]
  raised_by   uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  raised_from text, -- 'meeting','telegram','daily_brief','manual'
  source_meeting_id uuid REFERENCES meetings(id) ON DELETE SET NULL,
  status      text NOT NULL DEFAULT 'open' CHECK (status IN ('open','decided','deferred','dropped')),
  decision    text,
  decided_at  timestamptz,
  decided_by  uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_decisions_open ON decision_queue(company_id, status, created_at DESC) WHERE status = 'open';

-- 12 ----- capture_inbox -----
-- "Hey, capture this idea" voice or text → AI parses → creates task or decision.
CREATE TABLE capture_inbox (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
user_id      uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
source       text NOT NULL CHECK (source IN ('telegram','voice','web','email')),
raw_text     text,
raw_voice_url text,           -- link to Storage bucket
transcript   text,           -- whisper output
parsed_intent text,          -- 'task','decision','idea','note'
parsed_payload jsonb DEFAULT '{}::jsonb,
resolved_task_id uuid REFERENCES tasks(id) ON DELETE SET NULL,
resolved_decision_id uuid REFERENCES decision_queue(id) ON DELETE SET NULL,
status       text NOT NULL DEFAULT 'pending' CHECK (status IN ('pending','parsed','resolved','dropped')),
created_at   timestampz NOT NULL DEFAULT now(),
resolved_at   timestampz

);
CREATE INDEX idx_capture_user_pending ON capture_inbox(user_id, status, created_at DESC);

-- 13 ----- voice_memos -----
-- Standalone voice files (not just capture_inbox source). Whisper-transcribed.
CREATE TABLE voice_memos (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  storage_path text NOT NULL,      -- voice_memos/{user_id}/{uuid}.ogg
  duration_seconds int,
  transcript   text,
  transcript_lang text DEFAULT 'en',
  source       text CHECK (source IN ('telegram','web','manual')),
  related_capture_id uuid REFERENCES capture_inbox(id) ON DELETE SET NULL,
  created_at   timestampz NOT NULL DEFAULT now()
);

--
-- PHASE 2 : INTELLIGENCE (Week 2, Mon-Tue)
-- AI-generated content. Daily briefs, meeting prep docs, KPIs.
--

-- 14 ----- meeting_prep -----
-- Output of HIGH #141 — the AI prep doc generator. One row per meeting.
CREATE TABLE meeting_prep (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  calendar_event_id uuid NOT NULL REFERENCES calendar_events(id) ON DELETE CASCADE,
  generated_at timestampz NOT NULL DEFAULT now(),
  attendee_summary text,         -- "You last met Jane 2024-12-03 about X"
  open_threads  jsonb DEFAULT '{}::jsonb, -- [{title, decision_id, status}]
  related_tasks  jsonb DEFAULT '{}::jsonb,
  context_blocks jsonb DEFAULT '{}::jsonb, -- [{heading, body, source}]
  recommended_questions text[],
  full_brief_md text,           -- the rendered markdown brief
  delivered_via text[],         -- ['telegram','dashboard']
  UNIQUE (calendar_event_id)
);
CREATE INDEX idx_prep_event ON meeting_prep(calendar_event_id);

-- 15 ----- daily_briefs -----
-- 6am Telegram pulse. One row per user per day.
CREATE TABLE daily_briefs (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  brief_date  date NOT NULL,
  meetings_today jsonb DEFAULT '{}::jsonb,
  kpi_pulse     jsonb DEFAULT '{}::jsonb, -- {company_slug: {metric: {today, yesterday, ya, delta}}}
  blockers      jsonb DEFAULT '{}::jsonb,
  decisions_pending jsonb DEFAULT '{}::jsonb,
  body_md      text,
  delivered_at timestampz,
  created_at   timestampz NOT NULL DEFAULT now(),
  UNIQUE (user_id, brief_date)
);

-- 16 ----- kpis -----
-- Definition of each KPI per company. Source (mysql query / manual / claude calc).
CREATE TABLE kpis (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  slug        text NOT NULL,      -- 'gross_revenue_daily','membership_count'
  display_name text NOT NULL,
  unit        text,              -- '$','count','%'
  source_type  text NOT NULL CHECK (source_type IN ('littletree_mysql','manual','claude_calc','external_api')),
  source_query text,             -- SQL against littletree if applicable
  refresh_cadence text NOT NULL DEFAULT 'daily', -- 'daily','hourly','manual'
  display_order int DEFAULT 100,
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (company_id, slug)
);

-- 17 ----- kpi_snapshots -----
-- Time-series. One row per kpi per day (or hour). Charts read from here.
CREATE TABLE kpi_snapshots (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  kpi_id      uuid NOT NULL REFERENCES kpis(id) ON DELETE CASCADE,
  snapshot_date date NOT NULL,
  snapshot_hour int CHECK (snapshot_hour BETWEEN 0 AND 23),
  value       numeric NOT NULL,
  metadata    jsonb DEFAULT '{}::jsonb,
  created_at  timestampz NOT NULL DEFAULT now(),
  UNIQUE (kpi_id, snapshot_date, snapshot_hour)
);
CREATE INDEX idx_kpi_snap_kpi_time ON kpi_snapshots(kpi_id, snapshot_date DESC);

--
-- PHASE 2 : CROSS-VENTURE (Week 1 Fri)
-- JWT signing for the iframe bridge, ticker activity feed.
--

-- 18 ----- jwt_keys -----
-- Public keys for cross-venture verifiers (HIGH #137).
CREATE TABLE jwt_keys (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  kid         text UNIQUE NOT NULL, -- key id surfaced in JWT header
  algorithm   text NOT NULL DEFAULT 'RS256',
  public_key_pem text NOT NULL,
  private_key_pem text NOT NULL,    -- encrypted at rest, server-only access
  is_active   boolean NOT NULL DEFAULT true,
  rotated_at  timestampz,
  created_at  timestampz NOT NULL DEFAULT now()
);

-- 19 ----- venture_activity -----
-- Hover tooltip data on the venture cards (HIGH #138). Last 5 events per venture.
-- Materialized view candidate later — for now plain table populated by trigger.
CREATE TABLE venture_activity (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  event_type  text NOT NULL,        -- 'meeting','task_done','kpi_jump','decision_made'
  summary     text NOT NULL,
  occurred_at timestampz NOT NULL,
  related_entity_type text,
  related_entity_id uuid,
```

```
    created_at    timestamptz NOT NULL DEFAULT now()
  );
CREATE INDEX idx_venture_activity_recent ON venture_activity(company_id, occurred_at DESC);
```

– PHASE 2 : MENTORS / RAG (Week 2, Wed–Fri)
– PDF ingestion, semantic search, Ask Your Mentor.

```
– 20 ————— mentor_resources —————
– Top-level container per mentor. PDFs uploaded → broken into chunks → embedded.
CREATE TABLE mentor_resources (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  mentor      text NOT NULL CHECK (mentor IN ('tony','jay','strategic_coach','a360','ai_bail')),
  title       text NOT NULL,
  resource_type text NOT NULL CHECK (resource_type IN ('pdf','transcript','video','note')),
  storage_path text,           -- pdfs/{mentor}/{uuid}.pdf
  source_url  text,
  uploaded_by uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  page_count  int,
  metadata    jsonb DEFAULT '{}':jsonb,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_mentor_resources_mentor ON mentor_resources(mentor);
```

```
– 21 ————— pdf_chunks —————
– Chunked + embedded text. Powers Cmd+K search and Ask Your Mentor.
CREATE TABLE pdf_chunks (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  resource_id uuid NOT NULL REFERENCES mentor_resources(id) ON DELETE CASCADE,
  page_number int,
  chunk_index int NOT NULL,
  content     text NOT NULL,
  embedding   vector(1536),      -- OpenAI text-embedding-3-small
  token_count int,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_chunks_resource ON pdf_chunks(resource_id);
– HNSW for fast cosine search; build after backfill for speed.
– CREATE INDEX idx_chunks_embedding ON pdf_chunks USING hnsw (embedding vector_cosine_ops);
```

```
– 22 ————— skills —————
– Downloadable .skill modules. Surfaced on the Skills page + recommendation engine.
CREATE TABLE skills (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  slug        text UNIQUE NOT NULL,
  title       text NOT NULL,
  description  text,
  storage_path text NOT NULL,      -- skills/{slug}.skill
  category    text,
  embedding   vector(1536),      -- for recommendation match
  download_count int NOT NULL DEFAULT 0,
  is_active   boolean NOT NULL DEFAULT true,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);
```

```
– 23 ————— skill_downloads —————
CREATE TABLE skill_downloads (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  skill_id    uuid NOT NULL REFERENCES skills(id) ON DELETE CASCADE,
  user_id     uuid REFERENCES auth.users(id) ON DELETE SET NULL,
  source      text,             -- 'browse','recommendation','search'
  downloaded_at timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_skill_dl_user ON skill_downloads(user_id, downloaded_at DESC);
```

– PHASE 2 : SNAPSHOTS (Week 3, Mon–Wed)
– Weekly rhythm artefacts. PM weekly synthesis, Leah wins, status board.

```
– 24 ————— weekly_snapshots —————
– One row per user per ISO week. Supports the 3-col PM row on Mike's hub.
CREATE TABLE weekly_snapshots (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  iso_year    int NOT NULL,
  iso_week    int NOT NULL CHECK (iso_week BETWEEN 1 AND 53),
  watch       jsonb DEFAULT '{}':jsonb,  -- things being monitored
  building    jsonb DEFAULT '{}':jsonb,  -- in-flight work
  next_up     jsonb DEFAULT '{}':jsonb,  -- queued
  ideas       jsonb DEFAULT '{}':jsonb,  -- captured but unscheduled
  blockers    jsonb DEFAULT '{}':jsonb,
  narrative_md text,                 -- HIGH #173 Claude-written summary
  generated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (user_id, iso_year, iso_week)
);
```

```
– 25 ————— leah_wins —————
– Daily win capture form → Friday digest cron.
CREATE TABLE leah_wins (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     uuid NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  win_date    date NOT NULL,
  body        text NOT NULL,
  category    text,                 -- Claude-classified
  metadata    jsonb DEFAULT '{}':jsonb,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_wins_user_date ON leah_wins(user_id, win_date DESC);
```

– PHASE 3 : DEFERRED (build only when actually needed)
– Don't create yet. Listed here so we don't lose track.

```
– · email_threads    — full Gmail/Workspace ingestion + thread context
– · email_attachments — PDF / docx auto-archived from emails
– · sop_documents    — SOP draft generator + version history
– · sop_versions     —
– · investor_briefs  — Phase 3 investor-facing rollups
– · anomaly_alerts   — KPI deviation detection table
– · cross_company_metrics — rollups across the LTV portfolio
– · stripe_events     — once Stripe sync is wired (Phase 1 plumbing #X)
– · linear_issues     — once Linear sync is wired
– · qb_sync_log       — QuickBooks polling history per company
```

– COMMON RLS PATTERN (apply per table that has company_id)
– Wired here for the company-scoped tables in Phase 1+.

```
ALTER TABLE calendar_events ENABLE ROW LEVEL SECURITY;
ALTER TABLE meetings      ENABLE ROW LEVEL SECURITY;
ALTER TABLE meeting_attendees ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE telegram_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE decision_queue ENABLE ROW LEVEL SECURITY;
ALTER TABLE capture_inbox ENABLE ROW LEVEL SECURITY;
ALTER TABLE voice_memos ENABLE ROW LEVEL SECURITY;
ALTER TABLE meeting_prep ENABLE ROW LEVEL SECURITY;
ALTER TABLE daily_briefs ENABLE ROW LEVEL SECURITY;
ALTER TABLE kpis ENABLE ROW LEVEL SECURITY;
ALTER TABLE kpi_snapshots ENABLE ROW LEVEL SECURITY;
ALTER TABLE venture_activity ENABLE ROW LEVEL SECURITY;
ALTER TABLE weekly_snapshots ENABLE ROW LEVEL SECURITY;
ALTER TABLE leah_wins ENABLE ROW LEVEL SECURITY;

-- Generic per-company read policy (extend per table as needed)
CREATE POLICY ris_cal_read ON calendar_events FOR SELECT USING (has_company_access(company_id) OR user_id = auth.uid());
CREATE POLICY ris_meet_read ON meetings FOR SELECT USING (has_company_access(company_id));
CREATE POLICY ris_tasks_read ON tasks FOR SELECT USING (has_company_access(company_id) OR assigned_to = auth.uid() OR created_by = auth.uid());
CREATE POLICY ris_dec_read ON decision_queue FOR SELECT USING (has_company_access(company_id));
CREATE POLICY ris_kpi_read ON kpis FOR SELECT USING (has_company_access(company_id));
CREATE POLICY ris_kpi_snap_read ON kpi_snapshots FOR SELECT USING (
  has_company_access((SELECT company_id FROM kpis WHERE kpis.id = kpi_snapshots.kpi_id))
);
CREATE POLICY ris_va_read ON venture_activity FOR SELECT USING (has_company_access(company_id));

-- Per-user policies
CREATE POLICY ris_tg_self ON telegram_messages FOR ALL USING (user_id = auth.uid());
CREATE POLICY ris_capture_self ON capture_inbox FOR ALL USING (user_id = auth.uid());
CREATE POLICY ris_voice_self ON voice_memos FOR ALL USING (user_id = auth.uid());
CREATE POLICY ris_brief_self ON daily_briefs FOR ALL USING (user_id = auth.uid());
CREATE POLICY ris_snap_self ON weekly_snapshots FOR ALL USING (user_id = auth.uid());
CREATE POLICY ris_wins_self ON leah_wins FOR ALL USING (user_id = auth.uid());

-- Mentor/skills tables: readable to any authenticated user, writable by admin.
ALTER TABLE mentor_resources ENABLE ROW LEVEL SECURITY;
ALTER TABLE pdf_chunks ENABLE ROW LEVEL SECURITY;
ALTER TABLE skills ENABLE ROW LEVEL SECURITY;
ALTER TABLE skill_downloads ENABLE ROW LEVEL SECURITY;
CREATE POLICY ris_mr_read ON mentor_resources FOR SELECT USING (auth.role() = 'authenticated');
CREATE POLICY ris_chunks_read ON pdf_chunks FOR SELECT USING (auth.role() = 'authenticated');
CREATE POLICY ris_skills_read ON skills FOR SELECT USING (auth.role() = 'authenticated');
CREATE POLICY ris_dl_self ON skill_downloads FOR INSERT WITH CHECK (user_id = auth.uid());
```

```
--
-- updated_at TRIGGERS — keep timestamps fresh on every UPDATE
--

CREATE OR REPLACE FUNCTION touch_updated_at()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$;

DO $$
DECLARE t text;
BEGIN
  FOR t IN SELECT unnest(ARRAY[
    'profiles','companies','calendar_events','tasks','decision_queue',
    'kpis','skills'
  ]) LOOP
    EXECUTE format(
      'CREATE TRIGGER trg_%I_touch BEFORE UPDATE ON %I FOR EACH ROW EXECUTE FUNCTION touch_updated_at();',
      t, t
    );
  END LOOP;
END $$;
```

```
--
-- STORAGE BUCKETS (run via Supabase Studio or storage API, not pure SQL)
--

-- pdfs      — mentor PDFs, email attachments
-- voice_memos — Telegram voice messages, web recordings
-- uploads   — generic uploads tied to capture_inbox
-- skills    — downloadable .skill modules
-- exports   — generated CSV/PDF reports
```

```
--
-- BUILD ORDER FOR ANDRE
-- Apply in this order so dependencies don't fail. Each block is one migration.
--

-- M001 · extensions + profiles + companies + company_access + has_company_access()
-- M002 · auth_audit + activity_log
-- M003 · calendar_events + meetings + meeting_attendees + telegram_messages
-- M004 · tasks + telegram_messages.task_id FK + decision_queue + capture_inbox + voice_memos
-- M005 · meeting_prep + daily_briefs + kpis + kpi_snapshots
-- M006 · jwt_keys + venture_activity
-- M007 · mentor_resources + pdf_chunks + skills + skill_downloads
-- M008 · weekly_snapshots + leah_wins
-- M009 · RLS policies (this file's RLS section)
-- M010 · touch_updated_at function + triggers
```

[Quoted text hidden]
--
[Quoted text hidden]

andre.guy.b@gmail.com <andre.guy.b@gmail.com>
To: Tiffanie Rothwell <tiffanie@mjmventures.ai>
Cc: Mike David <mikedavidamg@gmail.com>

Wed, May 6, 2026 at 6:34 AM

Hi,

This looks good to me.

We need to nail down database credentials such as User, Password for the application. I believe that we should use the same user and password for every database.

In case that we need to have different database setup information, ask Valera what he thinks but it may be interesting to put database info in here.

Server

Database name

User

Password

```
CREATE TABLE companies (  
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),  
  slug        text UNIQUE NOT NULL,      -- 'little-tree-gas', 'lt-capital', etc.  
  name         text NOT NULL,  
  parent_group text NOT NULL CHECK (parent_group IN ('ltv','mjm','medbox','kcf','lahore')),  
  external_url text,                    -- existing dashboard URL if any  
  is_active    boolean NOT NULL DEFAULT true,  
  created_at   timestamptz NOT NULL DEFAULT now(),  
  updated_at   timestamptz NOT NULL DEFAULT now()  
);
```

```
INSERT INTO companies (slug, name, parent_group) VALUES  
(  
  ('little-tree-gas','Little Tree Gas','ltv'),  
  ('lt-capital','LT Capital','ltv'),  
  ('lt-pay','LT Pay','ltv'),  
  ('lt-fuel','LT Fuel','ltv'),  
  ('ai-agency','Ai Agency','ltv'),  
  ('jayi','Jayi','ltv'),  
  ('mjm','MJM Ventures','mjm'),  
  ('medbox','MedBox','medbox'),  
  ('kcf','KCF','kcf'),  
  ('lahore','Lahore','lahore');
```

André Boudreault

[Quoted text hidden]

andre.guy.b@gmail.com <andre.guy.b@gmail.com>
To: Tiffanie Rothwell <tiffanie@mjmventures.ai>
Cc: Mike David <mikedavidamg@gmail.com>

Wed, May 6, 2026 at 7:07 AM

Hi,

Based on what I know regarding the Supabase server, this is what Claude recommended we do. I have changed the "Mike's world to "masterdash"

Plan: create the schema + grants, set role search paths so RLS policies can find has_company_access() unqualified, then run your script with a one-line search_path prefix on top so every table lands in masterdash (not public).

There's one watch-item I'll handle proactively: your RLS policies call has_company_access(company_id) without a schema prefix. When those policies fire under the authenticated role, that role needs masterdash in its search path or the function won't resolve. The setup block below fixes that.

Step 1 — Get the file onto the server

From your laptop (the directory where mikes-world-schema.sql lives):

```
bash  
scp mikes-world-schema.sql user@mjmSPACE.com:/tmp/mikes-world-schema.sql  
(Replace user@ with whatever you usually SSH in as.)  
Then SSH into the server.
```

Step 2 — Create schema, grants, role search paths

Paste this whole block into your SSH session on the server. Run it as supabase_admin (the actual superuser, same role we used last time when postgres couldn't grant):

```
bash  
docker exec -i supabase-db psql -U supabase_admin -d postgres <<SQL'  
-- 1. Create the schema  
CREATE SCHEMA IF NOT EXISTS masterdash;  
  
-- 2. Let the Supabase API roles see it  
GRANT USAGE ON SCHEMA masterdash TO anon, authenticated, service_role;  
GRANT ALL    ON SCHEMA masterdash TO postgres, supabase_admin;  
  
-- 3. Default privileges for tables/functions/sequences created later  
ALTER DEFAULT PRIVILEGES IN SCHEMA masterdash  
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES  TO authenticated;  
ALTER DEFAULT PRIVILEGES IN SCHEMA masterdash  
  GRANT SELECT          ON TABLES  TO anon;  
ALTER DEFAULT PRIVILEGES IN SCHEMA masterdash  
  GRANT EXECUTE         ON FUNCTIONS TO anon, authenticated, service_role;  
ALTER DEFAULT PRIVILEGES IN SCHEMA masterdash  
  GRANT USAGE, SELECT   ON SEQUENCES TO authenticated;  
  
-- 4. Add masterdash to the search path of the API roles, so RLS policies can find  
--   has_company_access() unqualified. (auth and public stay reachable too.)  
ALTER ROLE authenticated SET search_path = "$user", public, masterdash, auth;  
ALTER ROLE anon          SET search_path = "$user", public, masterdash, auth;
```

```
ALTER ROLE service_role SET search_path = "$user", public, masterdash, auth;
```

```
\echo === Verifying ===
\dn masterdash
SELECT rolname, rolconfig FROM pg_roles
WHERE rolname IN ('authenticated','anon','service_role');
SQL
```

You should see the schema listed and each role's rolconfig showing the new search_path={...}. If anything errors out, stop and paste me the output before continuing.

Step 3 — Run the schema script inside masterdash

```
bash
( echo "SET search_path = masterdash, auth, public;"; \
  cat /tmp/mikes-world-schema.sql ) | \
  docker exec -i supabase-db psql -U supabase_admin -d postgres -v ON_ERROR_STOP=1
That -v ON_ERROR_STOP=1 tells psql to bail on the first error instead of plowing through 600 lines. Healthy run prints a stream of CREATE EXTENSION / CREATE TABLE / CREATE INDEX / CREATE POLICY / INSERT
0 10 lines and exits with code 0.
If the vector extension complains (could not open extension control file), pgvector isn't installed in this Supabase image — let me know and we'll either install it or comment out the two pgvector-dependent tables
(mentor_resources, pdf_chunks) for now.
```

Step 4 — Quick sanity check

```
bash
docker exec -i supabase-db psql -U supabase_admin -d postgres <<'SQL'
\dt masterdash.*
SELECT slug, name, parent_group FROM masterdash.companies ORDER BY parent_group, slug;
SQL
You should see ~27 tables and 10 seed company rows (the LTV ventures + MJM, MedBox, KCF, Lahore).
```

Follow-up you'll want soon, but not today

To make masterdash queryable through the Supabase REST/GraphQL API (so your frontend can hit <https://mjmspace.com/rest/v1/companies> etc.), you'll need to add masterdash to PGRST_DB_SCHEMAS in /opt/supabase/supabase/docker/.env and restart the kong and rest containers. Tell me when you're ready and I'll walk you through that — it's a 2-minute change.

André Boudreault

514 647-7577

From: Tiffanie Rothwell <tiffanie@mjmvventures.ai>
Sent: Tuesday, May 5, 2026 2:21 PM
To: andre.guy.b@gmail.com
Cc: Mike David <mikedavidamg@gmail.com>

[Quoted text hidden]

[Quoted text hidden]